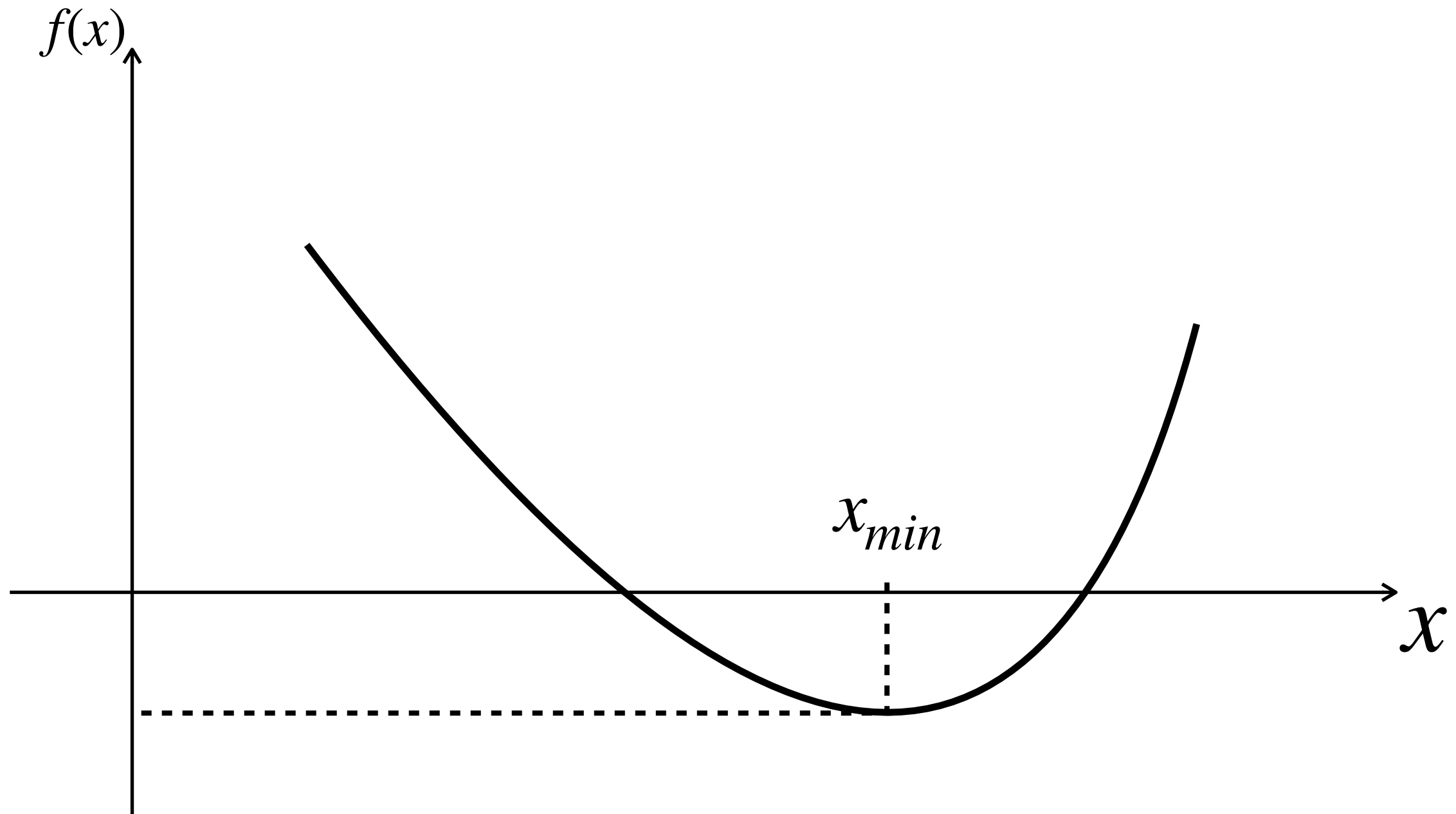


Optimisation

In many engineering problems, you are required to find the minimum of a function



- There are many methods/algorithms that can be used to find the minimum of a function.
- In this lecture we will be looking at the details of gradient based techniques.
- These techniques require information about the value of the function and its gradient. i.e. in order to find the location of the minimum of a function $f(x)$, we will need to be able to calculate df/dx and d^2f/dx^2 .
- We will first look at methods needing only information about the value of the function $f(x)$ and its gradient df/dx . Following this, we will look at methods requiring d^2f/dx^2 .

Standard Gradient Descent Method

- The standard gradient descent method uses the gradient of the function to obtain a better guess for the location of the local minimum of the function.
- The idea is that if you move in the opposite (-ve) direction of the gradient, the value of the function will always decrease.
- If you repeat this often enough, you will eventually reach the location of the *local minimum* of the function.
- In practice, you will want to move a fraction α of the gradient. α is called the learning rate.

- The update of equation for the standard gradient descent is given by

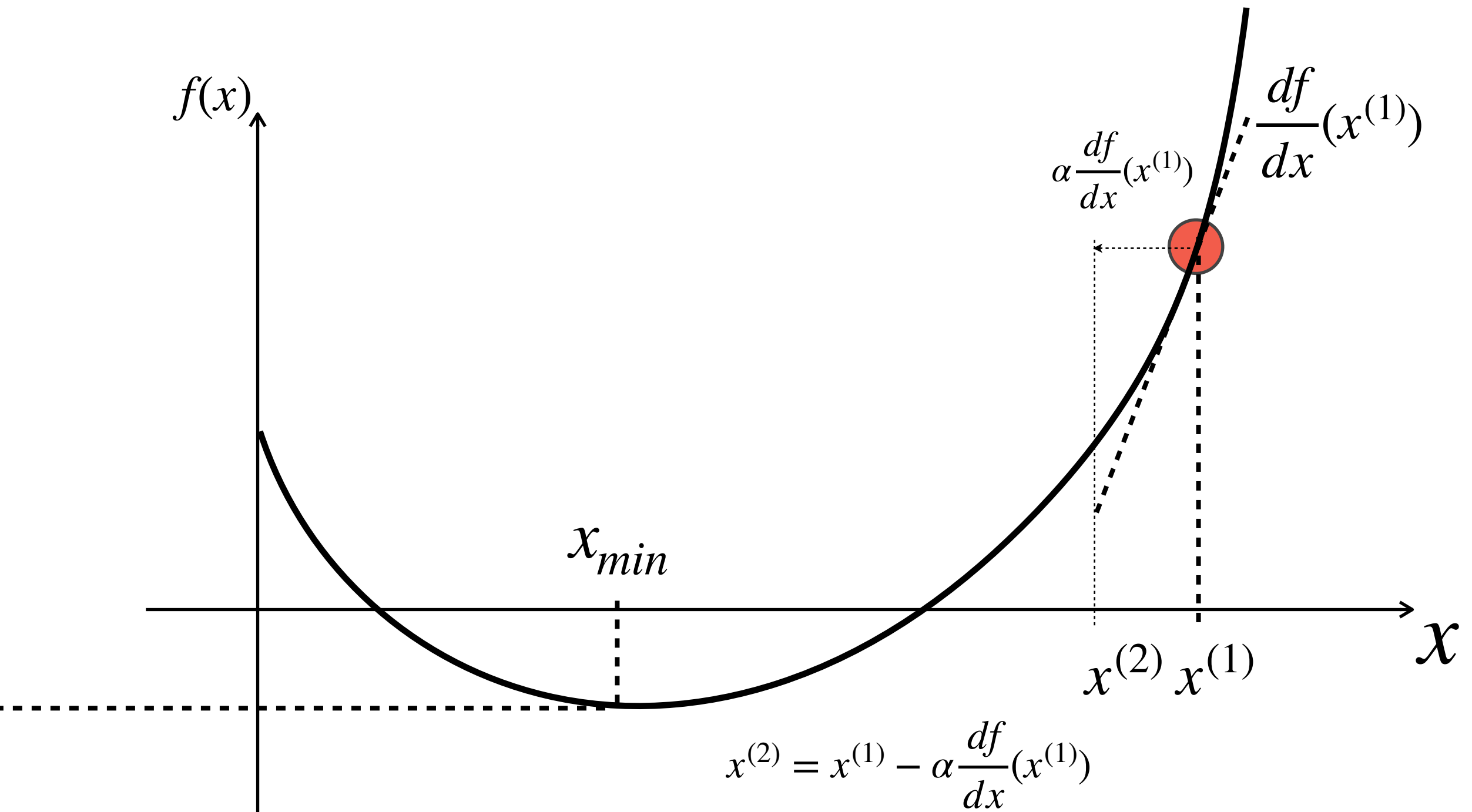
$$x^{(k+1)} = x^{(k)} - \alpha g^{(k)}$$

where

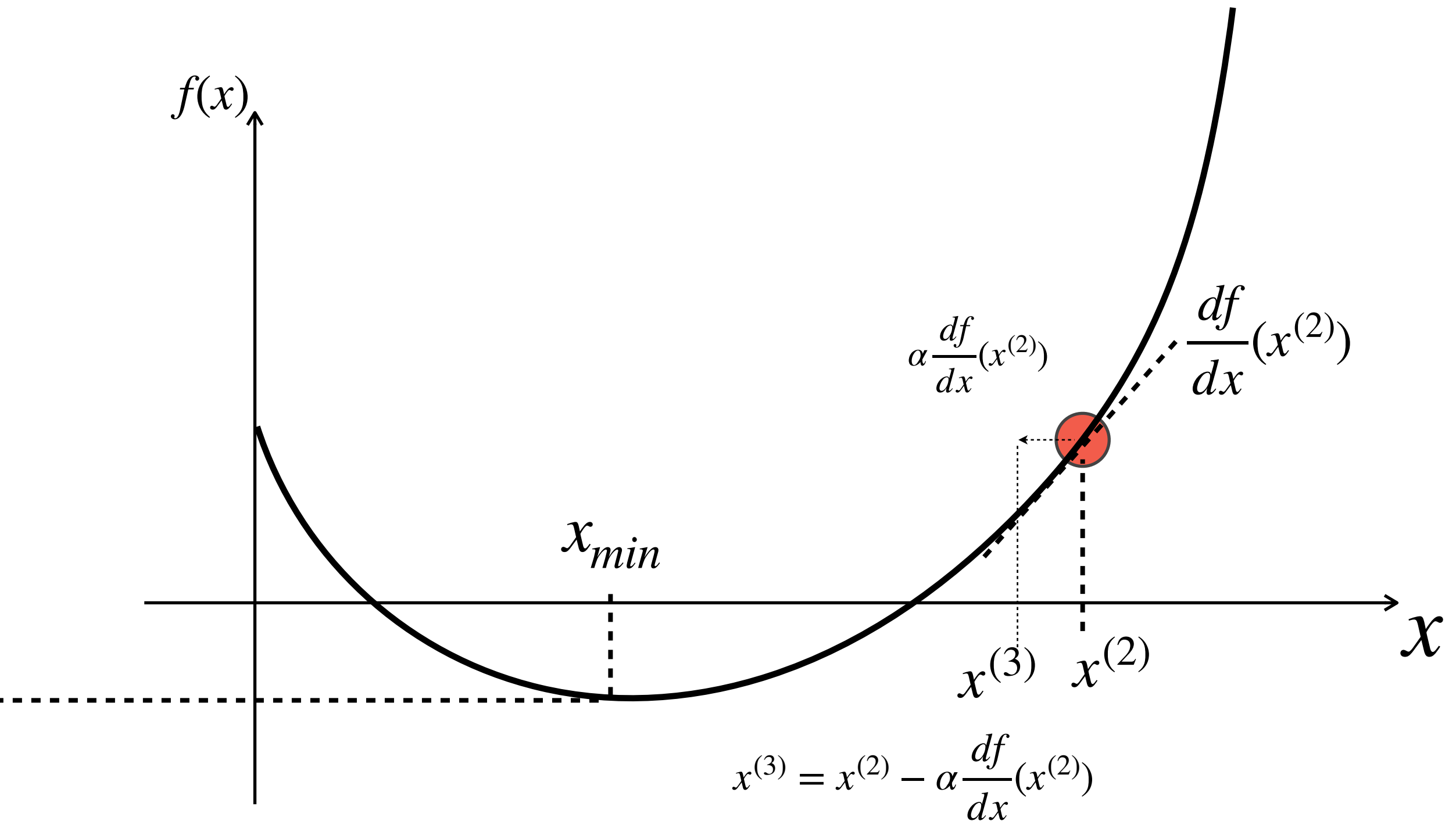
$g^{(k)} = \frac{df}{dx}(x^{(k)})$ is the gradient of the function $f(x)$ evaluated at the previous guess $x^{(k)}$. α is the learning rate and is usually given a small value (e.g. 0.1, 0.001). The superscript (k) indicates the k 'th iteration.

- Sometimes, you might have trouble converging to the minimum of the function. If that occurs, then usually decreasing the value of α might help.
- Choosing a good value of α is tricky. Too small, and it will take ages to converge to the minimum. Too big, and you might not get a solution!

Visualisation of Gradient Descent



Visualisation of Gradient Descent



Write a Julia program that uses the standard gradient descent method to find the minimum of the function

$$f(x) = x^2 - 2x + 0.6$$

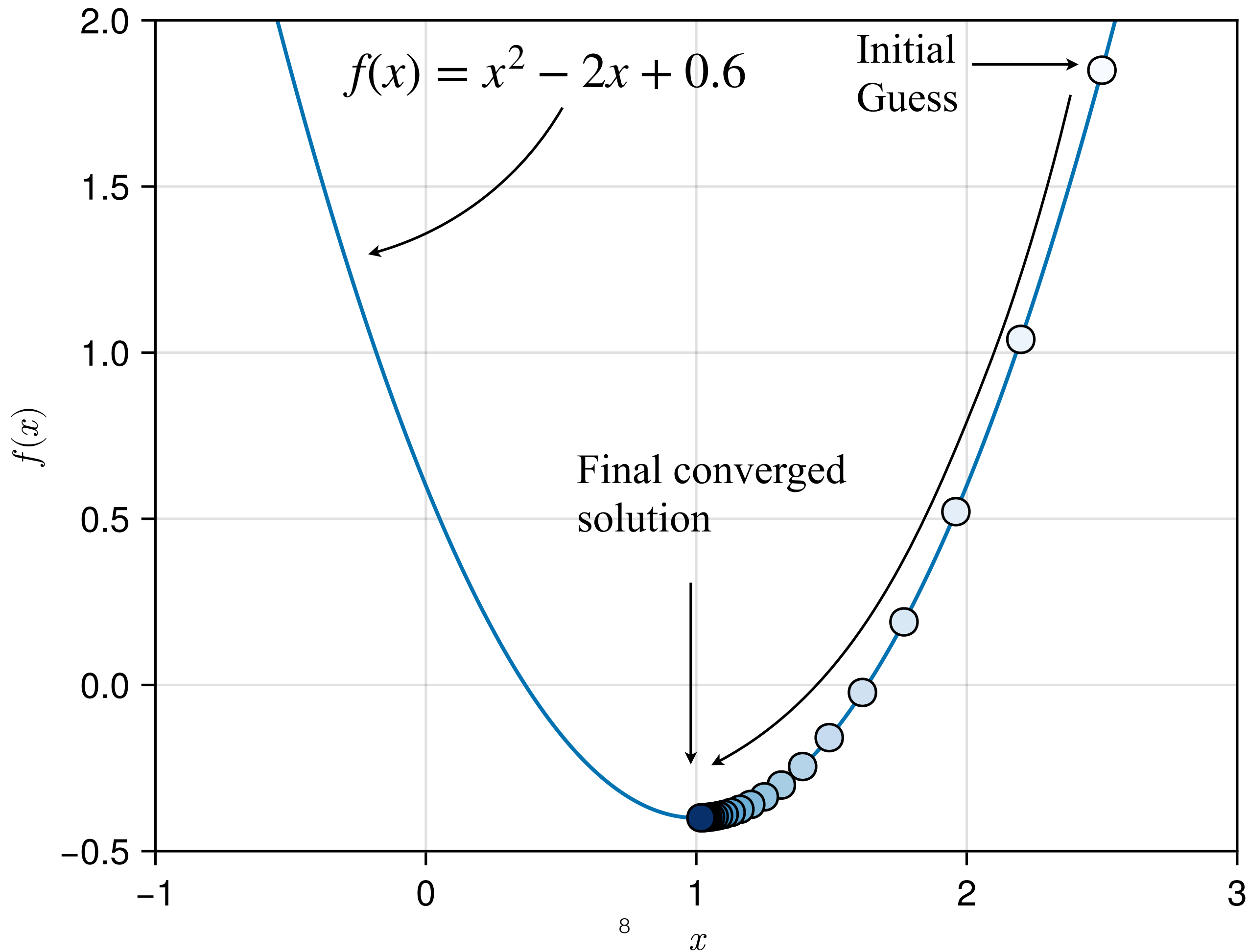
```
f(x)=x^2-2x+0.6 #Define the function
∇f(x)=2x-2 #Gradient of the function

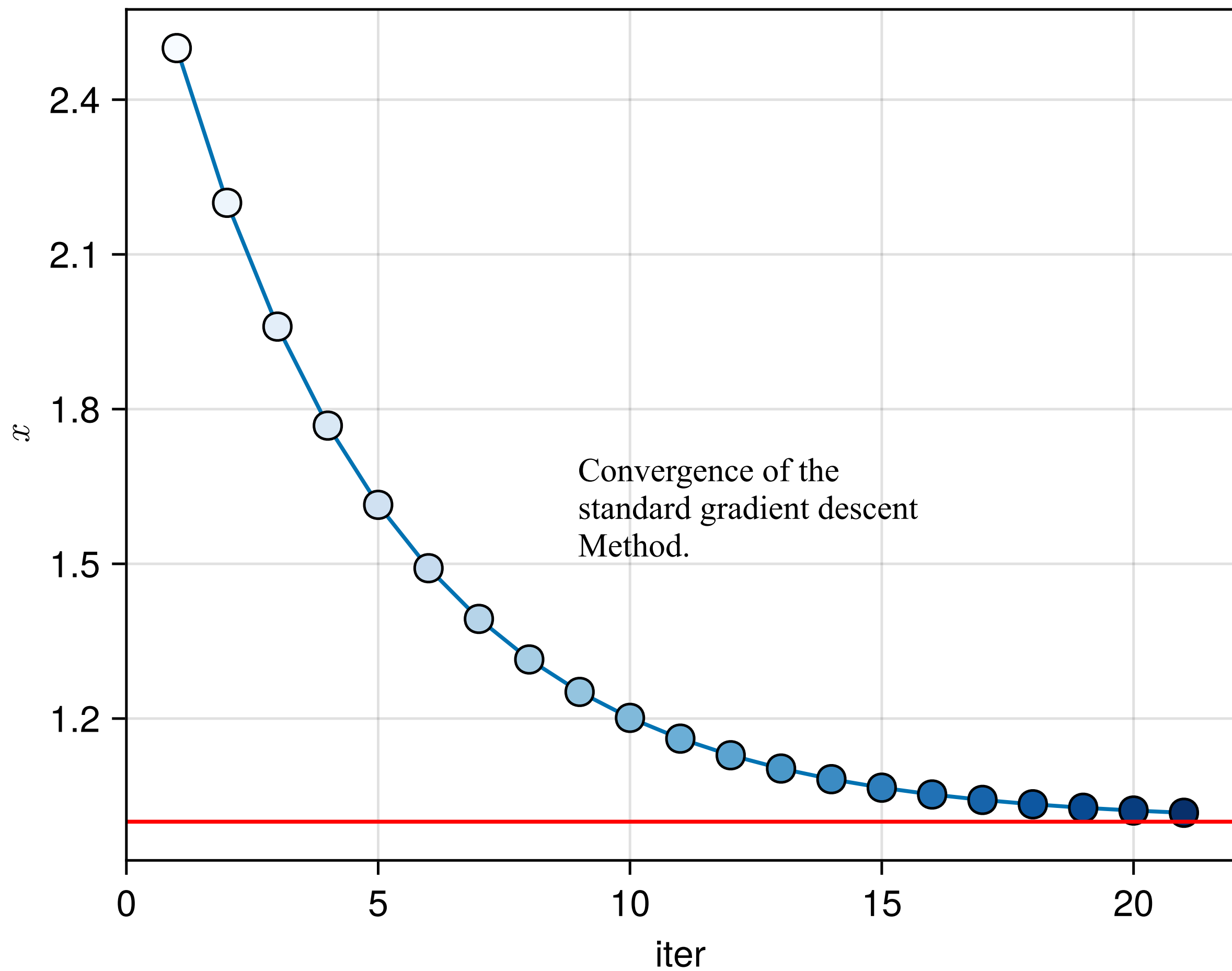
x0=2.5 #initial guess of x0
alpha=0.1 #learning rate
x=x0
for i in 1 : 20
    x-=alpha*∇f(x)
end
```

$$f(x) = x^2 - 2x + 0.6$$

$$g(x) = \frac{df(x)}{dx} = 2x - 2$$

$$x^{(k+1)} = x^{(k)} - \alpha g^{(k)}$$





Optimisation: Momentum

- One of the main problems with the standard gradient descent method is that it slows down quite a bit when we approach the minimum of the function
- This is usually not an issue for simple problems but for more complex problems involving lots of variables, we might have to wait ages before we obtain a good solution.
- The momentum method speeds up convergence, especially closer to the location of the minimum.
- The update equations for the momentum method is

$$v^{(k+1)} = \beta v^{(k)} - \alpha g^{(k)}$$

$$x^{(k+1)} = x^{(k)} + v^{(k+1)}$$

where

$g^{(k)} = \nabla f(x^{(k)})$, α is the learning rate and β is the momentum decay. The variable v can be thought of as the velocity. $v^{(0)}$ is usually set to be zero.

- It is insightful to write down the first few iterations of the momentum update equations

$$v^{(1)} = -\alpha g^{(0)}$$

$$x^{(1)} = x^{(0)} + v^{(1)}$$

$$v^{(2)} = \beta v^{(1)} - \alpha g^{(1)} = -\beta \alpha g^{(0)} - \alpha g^{(1)}$$

$$x^{(2)} = x^{(1)} + v^{(2)} = x^{(1)} - \beta \alpha g^{(0)} - \alpha g^{(1)}$$

$$v^{(3)} = \beta v^{(2)} - \alpha g^{(2)} = -\beta^2 \alpha g^{(0)} - \beta \alpha g^{(1)} - \alpha g^{(2)}$$

$$x^{(3)} = x^{(2)} + v^{(3)} = x^{(2)} - \beta^2 \alpha g^{(0)} - \beta \alpha g^{(1)} - \alpha g^{(2)}$$

- It is clear from the equations above that the momentum method remembers the past gradients. The higher the value of β , the more information it retains from the previous gradients.
- Note that if you set $\beta = 0$, you will get back the equations for the standard gradient descent method.
- The standard gradient descent only uses information about the local gradient. It "forgets" all the previous gradients. The momentum method retains information about all previous gradients.

Write a Julia program that uses the momentum method to find the minimum of the function

$$f(x) = x^2 - 2x + 0.6$$

$$f(x) = x^2 - 2x + 0.6$$

$$\nabla f(x) = 2x - 2$$

```
x0=2.5 #initial guess of x0
alpha=0.1 #learning rate
beta=0.6 #momentum
v=0.0 # initial velocity zero.
x=x0
for i in 1 : 20
    v=beta*v-alpha*\nabla f(x)
    x+=v
end
```

$$f(x) = x^2 - 2x + 0.6$$

$$g(x) = \frac{df(x)}{dx} = 2x - 2$$

$$v^{(k+1)} = \beta v^{(k)} - \alpha g^{(k)}$$

$$x^{(k+1)} = x^{(k)} + v^{(k+1)}$$

Optimisation: Nesterov Momentum

- Often, the momentum method overshoots the minimum point. This can be somehow circumvented by the Nesterov Momentum method.
- The Nesterov momentum method evaluates the gradient at the future projected location (rather than the current location).
- The update equations for the Nesterov momentum equations is

$$v^{(k+1)} = \beta v^{(k)} - \alpha g^{(k)}$$

$$x^{(k+1)} = x^{(k)} + v^{(k+1)}$$

where

$$g^{(k)} = \nabla f(x^{(k)} + \beta v^{(k)})$$

- The only different between the momentum Nesterov momentum update equations is that the gradient is now evaluated at $x^{(k)} + \beta v^{(k)}$ (for Nesterov Momentum) instead of $x^{(k)}$ (Momentum).

